

21강

자료구조(Data Structure)

&

Collection & Map

JDK로 프로그래밍 날개 달기



자료구조

(Data Structure)





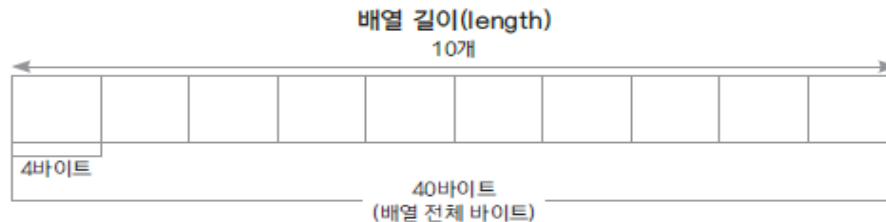
- 사전적인 의미는 **자료(Data)의 집합**의 의미하며, 각 원소들이 논리적으로 정의된 규칙에 의해 나열되며 자료에 대한 처리를 효율적으로 수행할 수 있도록 자료를 구분하여 표현한 것이라고 한다.
- 자료구조의 목적은 자료를 더 효율적으로 저장하고, 관리하기 위해 사용하며, 잘 선택된 자료구조는 실행시간을 단축시켜주거나 메모리 용량의 절약을 이끌어 낼 수 있다.
- 프로그램에서 사용할 많은 데이터를 메모리 상에서 어떤 구조를 가지고 관리 할 것인가 하는 여러 구현 방법을 말한다.
- 효율적인 자료구조가 성능 좋은 알고리즘의 기반이 된다.
- 자료의 효율적인 관리는 프로그램의 수행 속도와 밀접한 관련이 있다.
- 여러 자료 구조 중에서 구현하려는 프로그램에 맞는 최적의 자료구조를 활용해야 하므로 자료구조에 대한 이해가 중요하다.



➤ 선형 자료구조 (한 줄로 자료를 관리하기) – Array, LinkedList, Stack, Queue

• 배열 (Array) :

- 선형으로 자료를 관리하며 연속적 자료의 구조를 가진다,
- 정해진 크기의 메모리를 먼저 할당 받아 사용한다.
- 자료의 물리적 위치와 논리적 위치가 같다.
- 배열(Array)의 요소가 변하는 경우는 Array로 작성하는 건 비효율적이다.



• 연결 리스트 (LinkedList) :

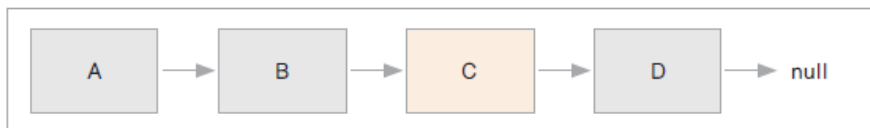
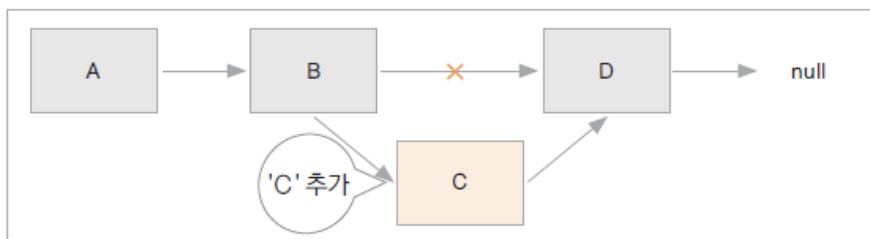
- 선형으로 자료를 관리하며, 자료가 추가될 때마다 메모리를 할당 받는다.
- 자료는 **Next Point(자기의 다음 주소를 가리키는 역할)**로 링크로 연결된다.
- 자료의 물리적 위치와 논리적 위치가 다를 수 있다.
- 배열(Array)의 단점을 보완 하였다.
- 배열(Array)보다 속도가 빠르다.
- 요소가 변하면 배열(Array) 보다 연결 리스트(LinkedList)를 사용하는 것이 더 효율적이다.



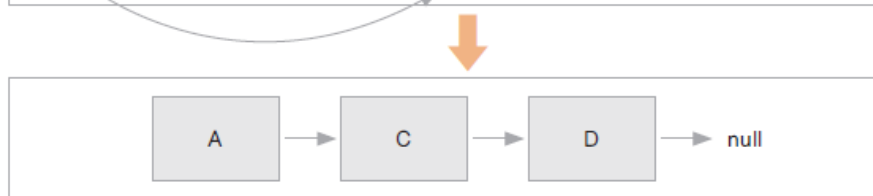
- 다음 요소에 대한 참조 값을 가지고 있음



- 리스트에 자료 추가하기

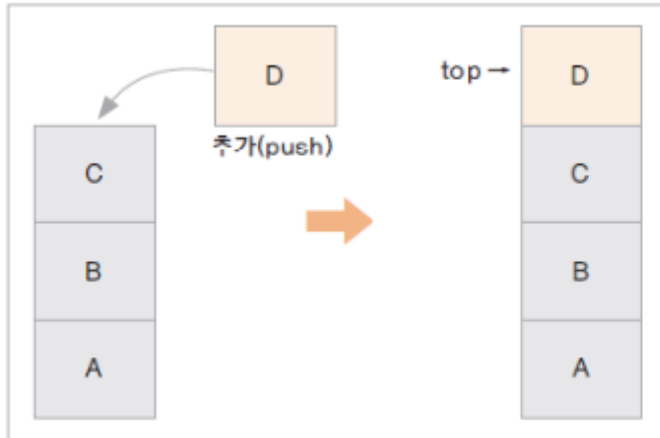


- 리스트에서 자료 삭제하기

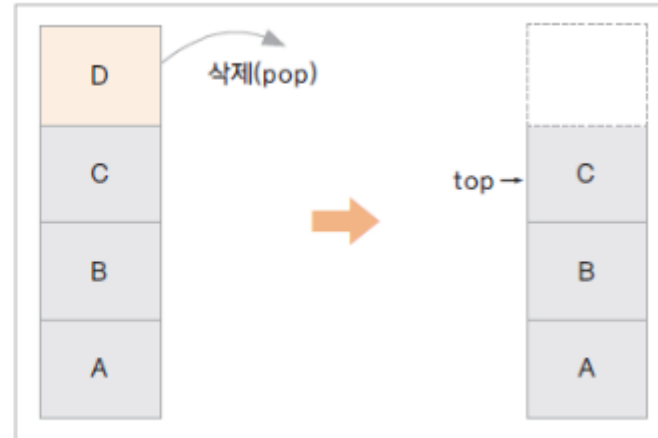




- **스택 (Stack)** : 가장 나중에 입력 된 자료가 가장 먼저 출력되는 자료 구조 (**Last In First Out**)

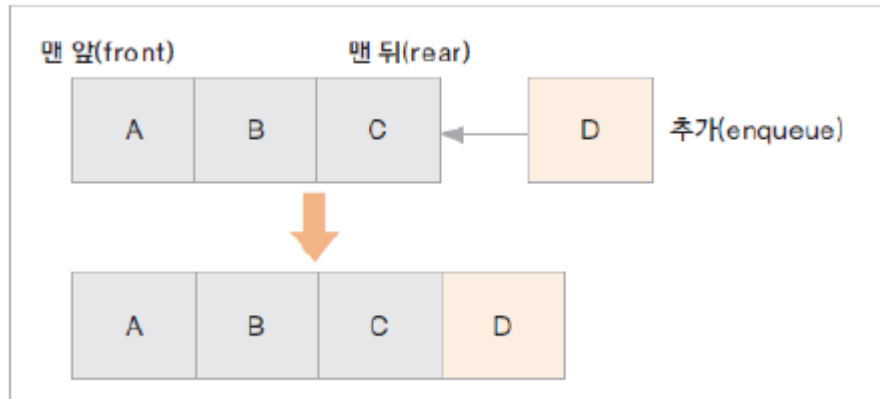


스택에 요소 추가(push)하기

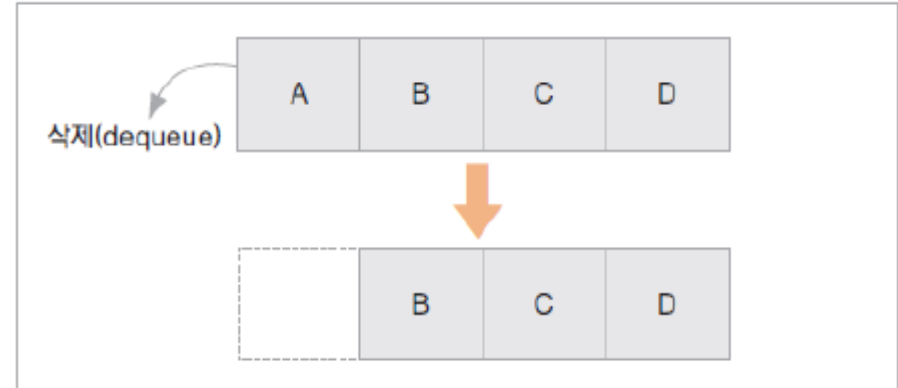


스택에서 요소 꺼내기(pop) 삭제하기

- **큐 (Queue)** : 가장 먼저 입력 된 자료가 가장 먼저 출력되는 자료 구조 (**First In First Out**)



큐에서 요소 추가(enqueue)하기

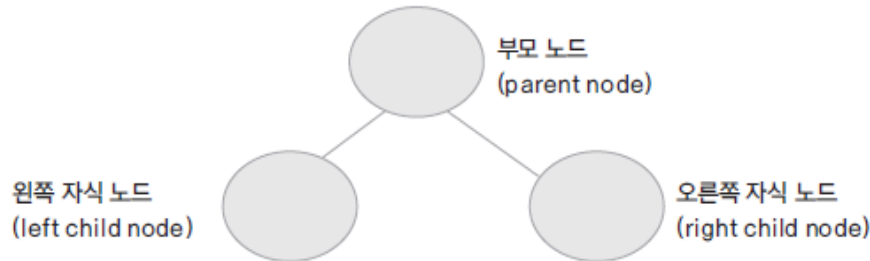


큐에서 요소 삭제(dequeue)하기

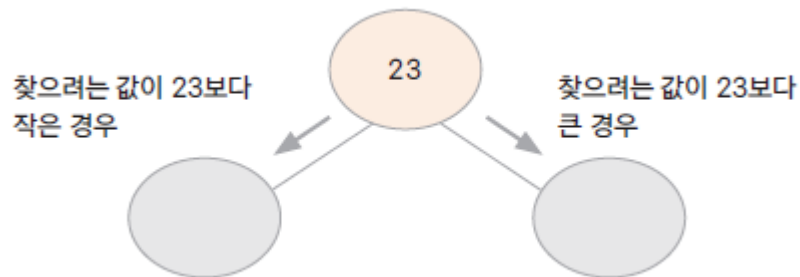


➤ 비선형 자료구조 - Tree, Graph

- 트리 (Tree) : 부모 노드와 자식 노드 간의 연결로 이루어진 자료 구조
- 이진 트리 (binary tree) : 부모노드에 자식노드가 두 개 이하인 트리



이진 검색 트리 (binary search tree)



트리 구조 특징

1. 자료(key)의 중복을 허용하지 않음
2. 왼쪽 자식 노드는 부모 노드보다 작은 값, 오른쪽 자식 노드는 부모 노드보다 큰 값을 가짐
3. 자료를 검색에 걸리는 시간이 평균 $\log(n)$ 임
4. **inorder traversal** 탐색을 하게 되면 자료가 정렬되어 출력됨 (Left - Root - Right)
5. 자료 검색을 위해 Tree를 사용 함

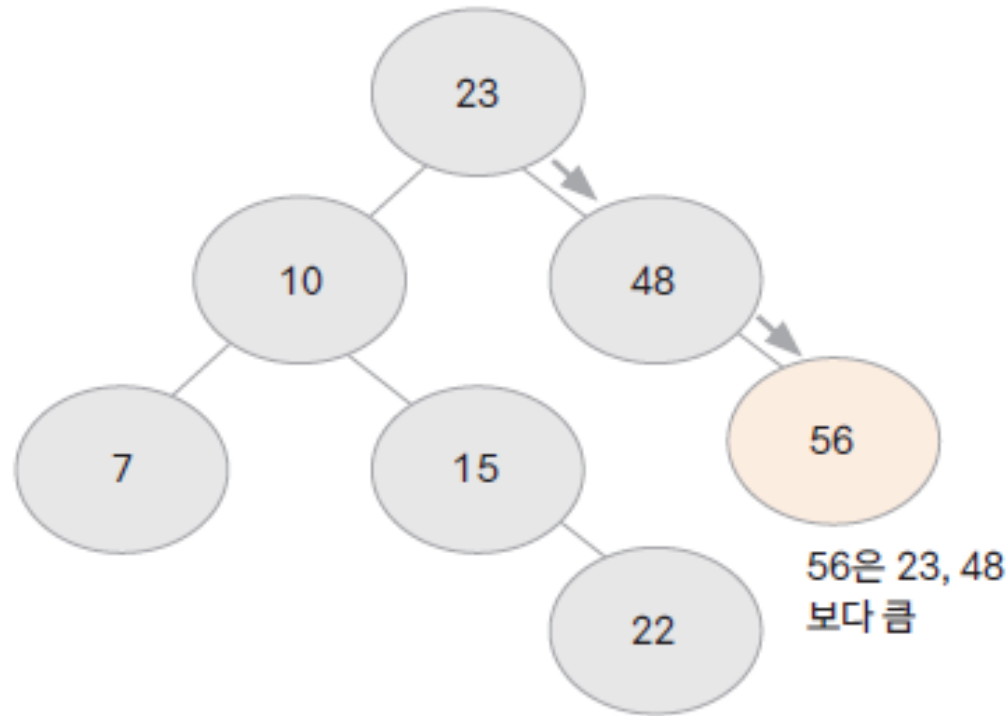
❖ **중위 순회(inorder traversal)**는 왼쪽 자식을 갖지 않는 노드에 도달할 때까지 왼쪽 서브트리를 따라간다. 조건에 만족하는 노드를 찾으면 해당 노드를 방문하고 부모 노드를 방문한 후 오른쪽 노드로 넘어가 왼쪽 서브트리를 따라가는 재귀적인 순회를 한다.

❖ **전위 순회(Preorder Traversal)** : Root - Left - Right

❖ **후위 순회(Postorder Traversal)** : Left - Right - Root



예) [23, 10, 48, 15, 7, 22, 56] 순으로 자료를 넣을 때 BST (binary search tree)



jdk 클래스 : TreeSet, TreeMap (Tree로 시작되는 클래스는 정렬을 지원 함)



- **해싱 (Hashing)** : 자료를 검색하기 위한 자료 구조 
- 산술연산을 이용한 검색을 위한 자료 구조
- 키(key)에 대한 자료를 검색하기 위한 사전(dictionary) 개념의 자료 구조
- key는 유일하고 이에 대한 value를 쌍으로 저장
- $\text{index} = h(\text{key})$: 해시 함수가 key에 대한 인덱스를 반환해 줌 해당 인덱스 위치에 자료를 저장하거나 검색하게 됨
- 해시 함수에 의해 인덱스 연산이 산술적으로 가능 $O(1)$
- 저장되는 메모리 구조를 **해시 테이블**이라 한다.

jdk 클래스 : HashMap, Properties

- 해시 테이블

버킷 M-1

1. 키(key)에 대한 value(값)가 있다.
2. 해시 키는 유일 해서 중복불가능하다.
3. 키(key)만 알면 value(값)을 꺼낼 수 없다.
4. 해시 Function에 의해 정해지기 때문에 순서와 상관이 없다.
5. $\text{Index} = h(\text{key}) \quad O(1)$

산술연산(+ - / * %)

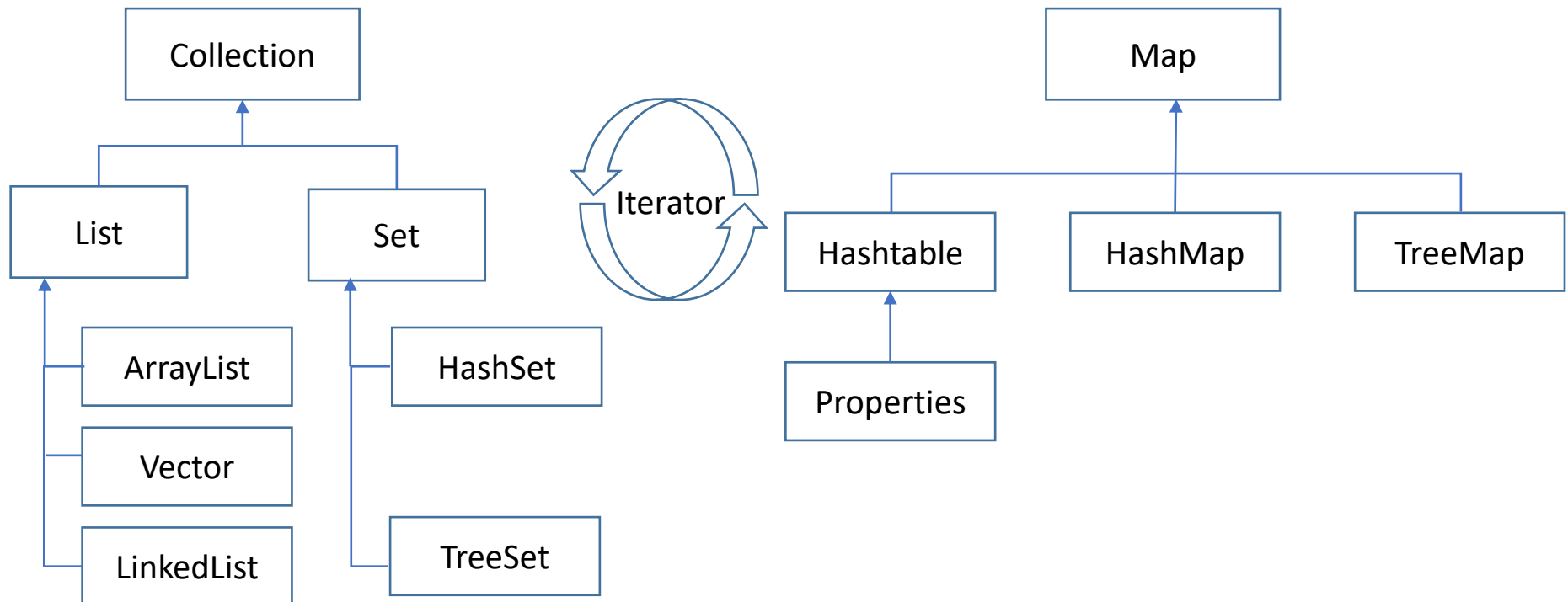
Collection & Map





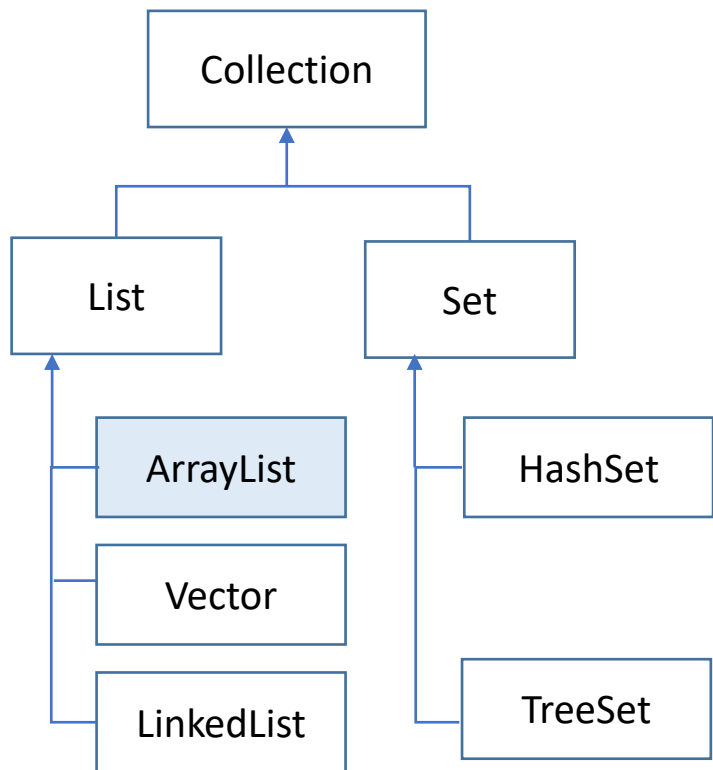
➤ 컬렉션(Collection) 프레임워크

- 프로그램 구현에 필요한 자료구조(Data Structure)를 구현해 놓은 라이브러리 이다.
- Java.util 패키지에 구현되어 있다.
- 개발에 소요되는 시간을 절약하면서 최적화 된 알고리즘을 사용할 수 있다.
- 여러 인터페이스와 구현 클래스 사용 방법을 이해해야 한다.
- 다수의 데이터를 쉽고 효과적으로 처리할 수 있는 표준화된 방법을 제공하는 클래스의 집합을 의미한다. 즉, 자료 구조와 알고리즘을 구조화하여 클래스로 구현해 놓은 것이다.





- 하나의 객체를 관리하기 위한 메서드가 선언된 인터페이스
- Collection은 요소들의 리스트나 집합을 관리하는 자료 구조로서 크기를 자동 조절하며 크기에 구애 받지 않고 요소의 추가, 삭제, 검색등을 쉽게 할 수 있어 배열보다 용이하다.
- 하위에 List와 Set 인터페이스가 있음



분류	설명
List 인터페이스	• 객체를 순서에 따라 저장하고 관리하는데 필요한 메서드가 선언된 인터페이스 • 자료구조 리스트 (배열, 연결리스트)의 구현을 위한 인터페이스 • 중복 허용 • 이 인터페이스를 구현한 클래스는 ArrayList, Vector, LinkedList, Stack, Queue 등
Set 인터페이스	• 순서와 관계없이 중복을 허용하지 않고 유일한 값을 관리하는데 필요한 메서드가 선언됨 • 아이디, 주민번호, 사번등을 관리하는데 유용 • 저장된 순서와 출력되는 순서는 다를 수 있음 • 이 인터페이스를 구현한 클래스는 HashSet, TreeSet 등



Collection 인터페이스에 선언된 주요 메서드

메서드	설명
Boolean <u>add(E e)</u>	Collection에 <u>객체를 추가</u> 합니다.
void <u>clear()</u>	Collection의 모든 <u>객체를 제거</u> 합니다.
<u>Iterator<E></u> Iterator	Collection을 순환할 <u>반복자(iterator)</u> 를 반환합니다.
Boolean <u>remove(Object o)</u>	Collection에 매개변수에 해당하는 인스턴스가 <u>존재하면 제거</u> 합니다.
Int <u>size()</u>	Collection에 <u>있는 요소의 개수</u> 를 반환합니다.
Object[] <u>toArray()</u>	해당 컬렉션의 모든 요소를 <u>Object 타입의 배열로 반환</u> 함
boolean <u>isEmpty()</u>	해당 컬렉션이 <u>비어있는지</u> 를 확인함.

Map 인터페이스 주요 메서드





➤ Map 인터페이스란 ?

- Map 컬렉션은 **키(key)와 값(value)**으로 구현된 Map.Entry 객체를 저장하는 구조를 가지고 있다.
- Entry는 Map 인터페이스 내부에 선언된 중첩 인터페이스 이다. 여기서 키와 값은 모두 객체이다.
- 키(key)는 중복 저장 될 수 없지만, 값(value)은 중복 저장 될 수 있다. 만약 기존에 저장된 키와 동일한 키로 값을 저장하면 기존의 값은 없어지고 새로운 값으로 **대체**된다.
- Map 컬렉션에는 HashMap, Hashtable, LinkedHashMap, Properties, TreeMap등이 있다.
- 키(key)를 이용하여 값(value)을 저장하거나 검색, 삭제 할 때 사용하면 편리하다.
- 키(key)가 되는 객체는 객체의 유일성 함의 여부를 알기 위해 **equals()와 hashCode()메소드**를 재정의한다.



➤ Map 인터페이스 메소드

기능	메서드	설명
<u>추가</u>	V put (K key, V value)	주어진 <u>키와 값을 저장</u> 합니다. 새로운 키일 경우 null을 반환하고, 동일한 키가 있을 경우 <u>기존 값을 대체</u> 하고 <u>이전 값을 반환</u> 합니다.
<u>조회</u>	boolean containsKey (Object key)	주어진 <u>키가 있는지 여부를 확인</u> 합니다. ← T/F
	boolean containsValue (Object value)	주어진 <u>값이 있는지 여부를 확인</u> 합니다.
	Set < Map.Entry<K, V> > entrySet()	키와 값 쌍으로 구성된 모든 Map.Entry 객체를 <u>Set에 담아서 반환</u> 합니다.
<u>검색</u>	V get (Object key)	<u>주어진 키가 있는 값을 반환</u> 합니다.
<u>검사</u>	boolean isEmpty ()	<u>컬렉션이 비어 있는지 여부를 확인</u> 합니다.
<u>조회</u>	Set <K> keySet()	모든 키를 <u>Set 객체에 담아서 반환</u> 합니다.
	int size ()	저장된 키의 <u>총 개수를 반환</u> 합니다.
	Collection <V> values()	저장된 모든 값을 <u>Collection에 담아서 반환</u> 합니다.
<u>삭제</u>	void clear ()	<u>모든 Map.Entry(키와 값)를 삭제</u> 합니다.
	V remove (Object key)	주어진 키와 일치하는 <u>Map.Entry를 삭제</u> 하고 <u>값을 반환</u> 합니다.



• [put(키, 값)]

- 키(Key)와 값(value)을 입력
- 제네릭으로 지정한 타입에 맞는 일반 배열을 이용해 한번에 넣어줄 수 있음

```
Map< k, v > map = new HashMap< k, v >;
```

키 타입 값 타입

```
public static void main(String[] args) {
```

```
    HashMap<String, String> list = new HashMap<String, String>();
```

```
    String[] key = { "a", "b", "c", "d", "e" };
```

```
    String[] value = { "apple", "banana", "candy", "dog", "enum" };
```

```
    for (int i = 0; i < key.length; i++)
```

```
        list.put(key[i], value[i]);
```

```
    // {a=apple, b=banana, c=candy, d=dog, e=enum}
```

```
    System.out.println(list);
```



Console X

<terminated> HashTest (1) [Java Application] C:\Users\Admin\p2\pool\plugins\org.eclipse

{a=apple, b=banana, c=candy, d=dog, e=enum}



- [**keySet()** / **values()**]

- 키 / 값을 별도로 추출하여 컬렉션 형태로 반환 (Set 생성)

```
public static void main(String[] args) {  
  
    HashMap<String, String> list = new HashMap<String, String>();  
  
    String[] key = { "a", "b", "c", "d", "e" };  
    String[] value = { "apple", "banana", "candy", "dog", "enum" };  
  
    for (int i = 0; i < key.length; i++)  
        list.put(key[i], value[i]);  
  
    // [a, b, c, d, e]  
    System.out.println(list.keySet());  
    // [apple, banana, candy, dog, enum]  
    System.out.println(list.values());  
}
```



```
Console x  
<terminated> HashTest (1) [Java Application] C:\Users\Admin#\br/>[a, b, c, d, e]  
[apple, banana, candy, dog, enum]
```



- [**get()**]

- 키 값을 제공하면 해당 값을 반환

```
public static void main(String[] args) {  
  
    HashMap<String, String> list = new HashMap<String, String>();  
  
    String[] key = { "a", "b", "c", "d", "e" };  
    String[] value = { "apple", "banana", "candy", "dog", "enum" };  
  
    for (int i = 0; i < key.length; i++)  
        list.put(key[i], value[i]);  
  
    System.out.println(list.get("c")); // candy
```



```
Console X  
<terminated> HashTest (1)  
candy
```



- [**replace()**]

- 키 값과 바꿀 값을 제공하면 해당 키의 값을 변경함

```
public static void main(String[] args) {  
  
    HashMap<String, String> list = new HashMap<String, String>();  
  
    String[] key = { "a", "b", "c", "d", "e" };  
    String[] value = { "apple", "banana", "candy", "dog", "enum" };  
  
    for (int i = 0; i < key.length; i++)  
        list.put(key[i], value[i]);  
  
    list.replace("c", "coin");  
    System.out.println(list.get("c")); // coin
```



```
Console ×  
<terminated> HashTest (1)  
coin
```



• [put()]

- 키 값과 추가할 값을 제공하면 새로운 "key : value"가 추가됨
- 만약 중복되는 키 값을 제공하면 기존 키가 가지고 있던 값을 제공된 값으로 변경함

```
public static void main(String[] args) {  
  
    HashMap<String, String> list = new HashMap<String, String>();  
  
    String[] key = { "a", "b", "c", "d", "e" };  
    String[] value = { "apple", "banana", "candy", "dog", "enum" };  
  
    for (int i = 0; i < key.length; i++)  
        list.put(key[i], value[i]);  
  
    list.put("c", "coin"); // 기존값으로 대체  
    System.out.println(list.get("c")); // coin  
  
    list.put("f", "float"); // 새로 추가됨  
    System.out.println(list.get("f")); // float
```



Console X	
<terminated> HashTest	
coin	
float	



- [**containsKey()** / **containsValue()**]

- key / value가 존재하는지 여부 확인 후 true / false 반환

```
HashMap<String, String> list = new HashMap<String, String>();
```

```
String[] key = { "a", "b", "c", "d", "e" };
```

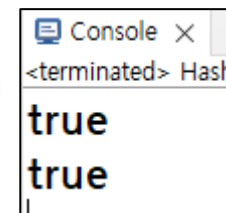
```
String[] value = { "apple", "banana", "candy", "dog", "enum" };
```

```
for (int i = 0; i < key.length; i++)
```

```
list.put(key[i], value[i]);
```

```
System.out.println(list.containsKey("a")); // true
```

```
System.out.println(list.containsValue("dog")); //true
```





- [remove() / clear()]

- remove() : 제공한 키 값의 항목을 제거
- clear() : 모두 삭제

```
HashMap<String, String> list = new HashMap<String, String>();
```

```
String[] key = { "a", "b", "c", "d", "e" };
```

```
String[] value = { "apple", "banana", "candy", "dog", "enum" };
```

```
for (int i = 0; i < key.length; i++)
```

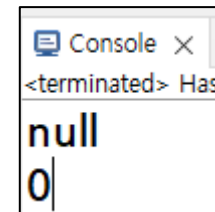
```
list.put(key[i], value[i]);
```

```
list.remove("d"); // "d" 키/값 삭제
```

```
System.out.println(list.get("d")); // null
```

```
list.clear(); // 모두 삭제
```

```
System.out.println(list.size()); // 남은 갯수 0
```





- 확장 for문을 이용한 출력

```
HashMap<String, String> list = new HashMap<String, String>();
```

```
String[] key = { "a", "b", "c", "d", "e" };
```

```
String[] value = { "apple", "banana", "candy", "dog", "enum" };
```

```
for (int i = 0; i < key.length; i++)
```

```
list.put(key[i], value[i]);
```

```
for (String keys : list.keySet())
```

```
System.out.println(keys + " : " + list.get(keys));
```

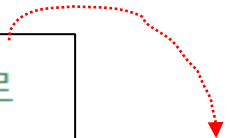


```
Console X
<terminated> HashTest (
a : apple
b : banana
c : candy
d : dog
e : enum
```



- 객체 순회 하기
- Iterator() 메서드는 하나의 Collection 객체만을 반환하므로 pair로 이루어진 객체는 각각 순회 해야 함

```
// Iterator() 메서드는 하나의 Collection 객체만을 반환하므로
// pair로 이루어진 객체는 각각 순회 해야 함
// Iterator을 이용한 출력
Set<String> keySet = list.keySet();
Iterator<String> ir = keySet.iterator();
while (ir.hasNext()) { // 다음 key가 있으면
    String k = ir.next();           // key는 String
    String v = list.get(k);         // value 꺼내기
    System.out.println(k + " : " + v);
}
```



```
a : apple
b : banana
c : candy
d : dog
e : enum
```



```
Iterator를 사용해 전체 list를
출력
```



- Map 인터페이스를 구현한 클래스 중 **가장 일반적으로 사용하는 클래스**
- Hashtable 클래스는 자바2 부터 제공된 클래스로 Vector 처럼 동기화를 제공 함
- 여러 메서드를 활용하여 pair 자료를 쉽고 빠르게 관리 할 수 있음



➤ HashMap 메모리 구조

1. 메모리 저장 위치

- **힙(Heap) 메모리에 저장**됩니다
- nations 변수는 스택에 있지만, 실제 HashMap 객체와 데이터는 힙에 생성됩니다

2. 내부 구조

- HashMap은 **배열 + 연결 리스트(또는 트리)** 조합으로 동작합니다:

3. 저장 과정:

Key

`nations.put("korea", 5000);` // 이렇게 저장하면...

- "korea".hashCode() 호출 → 해시값 생성
- 해시값을 배열 크기로 나눈 나머지로 인덱스 계산
- 해당 인덱스의 버킷에 Node(key, value, hash, next) 저장

4. Map이 비선형인 이유 vs Tree 구조

- HashMap은 비선형이지만 Tree 구조가 아닙니다
- 비선형 = 순서가 없다는 의미 (입력 순서 ≠ 저장 순서)

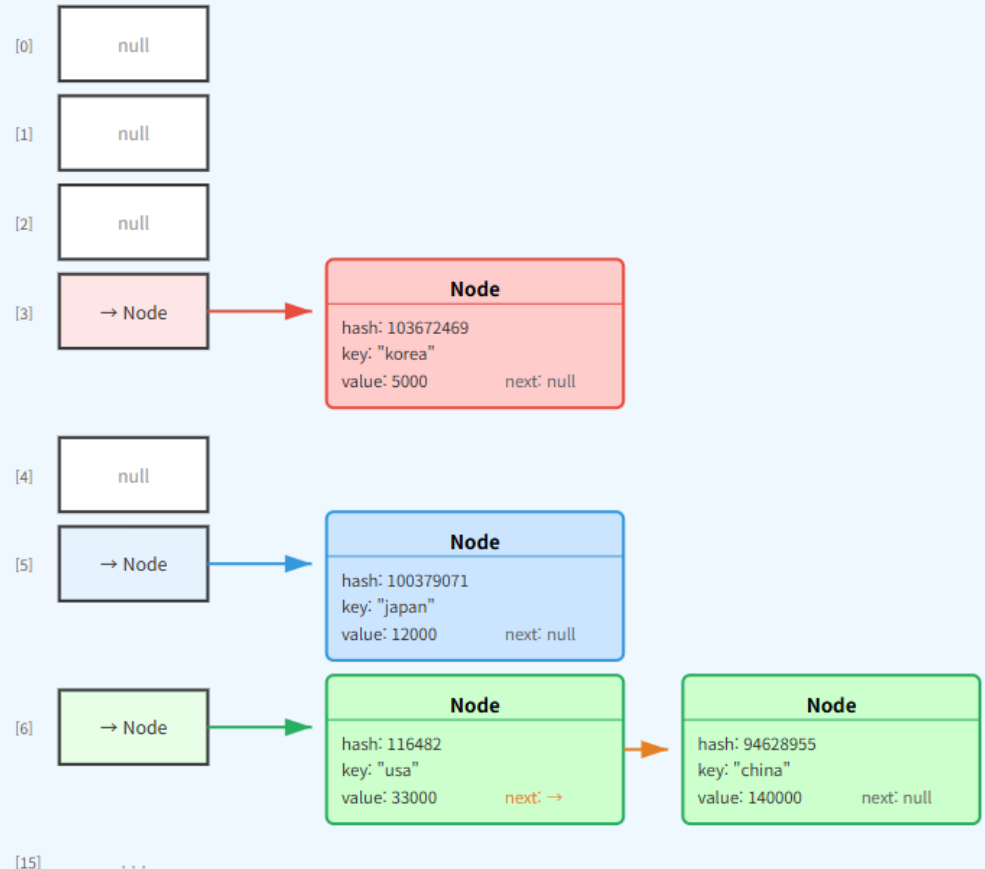
5. HashMap의 실제 구조:

- 기본: 배열 + LinkedList (Separate Chaining 방식)
- 충돌이 많을 때 (Java 8+): LinkedList → Red-Black Tree로 자동 변환
 - 체인 길이 ≥ 8 개: Tree로 변환
 - 체인 길이 ≤ 6 개: 다시 LinkedList로 변환

Heap Memory - HashMap 객체

내부 배열 (Bucket Array)

기본 크기: 16 (capacity)



해시 충돌 처리 (Separate Chaining)

같은 인덱스에 여러 값이 매핑되면 LinkedList 형태로 연결됩니다.
Java 8+에서는 체인이 길어지면 (8개 이상) 자동으로 Tree 구조로 변환됩니다.

연습 문제



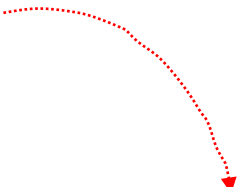


➤ StringTokenizer 클래스란?

- **StringTokenizer** 클래스는 하나의 문자열을 사용자가 지정한 구분자(delimiter)를 기준으로 여러 개의 문자열로 분리(**토큰화**)하는 클래스이다. 이때 분리된 각각의 문자열을 토큰(Token)이라고 한다.
- String : 문자열
- Tokenizer : 토큰화 한다.



```
public static void main(String[] args) {  
    // StringTokenizer 생성방법 3가지  
    String str = "java android";  
    String str1 = "java!android ";  
    String str2 = "java!android-fun";  
  
    StringTokenizer st1 = new StringTokenizer(str);  
    // nextToken() : 객체에서 다음토큰을 반환  
    System.out.println(st1.nextToken());  
    System.out.println(st1.nextToken());  
  
    StringTokenizer st3 = new StringTokenizer(str1,"!");  
    for(int i=0;i<2;i++) {  
        System.out.println(st3.nextToken());  
    }  
    // 여러개 구분자를 자를때  
    StringTokenizer st4 = new StringTokenizer(str2,"!-");  
    for(int i=0;i<3;i++) {  
        // st4.nextToken().trim() : 혹시 모를 공백을 제거  
        System.out.println(st4.nextToken().trim());  
    }  
    // true를 입력하면 구분자로 지정된 ! 또한 토큰으로 넣어준다.  
    StringTokenizer st5 = new StringTokenizer(str1,"!",true);  
    for(int i=0;i<3;i++) {  
        System.out.println(st5.nextToken());  
    }  
}
```



```
Outline Task List Console X  
<terminated> StringTokenTest [Java Application]  
java  
android  
-----  
java  
android  
-----  
java  
android  
fun  
-----  
java  
!  
android  
-----
```



Package 명 hashMapEx 로 작성 할 것

Class 명 : HashMapEx01

문제1> '그만'이라고 입력될 때까지 나라 이름과 인구를 입력 받아 저장하고, 다시 나라 이름을 입력 받아 인구를

출력하는 프로그램을 작성하시오.

인구 검색 프로그램

```
HashMap<String, Integer> nations = new HashMap<String, Integer>();
```

```
나라 이름과 인구를 입력하세요.(예: korea 5000
나라 이름, 인구 >> korea 5000
나라 이름, 인구 >> usa 6000
나라 이름, 인구 >> 그만
인구 검색 >> korea
korea의 인구는 5000
인구 검색 >> usa
usa의 인구는 6000
인구 검색 >> 프랑스
프랑스나라는 없습니다.
인구 검색 >> 그만
인구 검색 종료
```




Package 명 hashMapEx 로 작성 할 것

Class 명 : HashMapEx02, Location

문제2> 도시 이름, 위도, 경도 정보를 가진 Location 클래스를 작성하고, 도시 이름을 '키' 로 하는 HashMap<String, Location> 컬렉션을 만들고, 사용자로 부터 입력 받아 4개의 도시를 저장하라, 그리고 도시 이름으로 검색하는 프로그램을 작성하시오.

도시, 경도, 위도를 입력 하시오.

>> 서울,37,126
>> LA,34,-118
>> 파리,2,48
>> 시드니,151,-33

서울 37 126
LA 34 -118
파리 2 48
시드니 151 -33

도시 이름 >> 서울
서울 37 126
도시 이름 >> 프랑스
프랑스는 없습니다.
도시 이름 >> 그만
정보 검색 종료



Package 명 hashMapEx 로 작성 할 것

Class 명 : HashMapEx03

문제3> 이름과 학점(4.5 만점)을 5개 입력 받아 HashMap에 저장하고, 장학생 선발 기준을 입력 받아 장학생 명단을 출력 하시오.

미래 장학금 관리 시스템 입니다.
이름과 학점 >> 적당히 3.1
이름과 학점 >> 나탈락 2.4
이름과 학점 >> 최고조 4.3
이름과 학점 >> 상당히 3.9
이름과 학점 >> 고득점 4.0
장학생 선발 학생 기준 입력 >>3.2
장학생 명단 :최고조 상당히 고득점



Package 명 hashMapEx 로 작성 할 것

Class 명 : HashMapEx04

문제4> 고객의 이름과 포인트 점수를 관리하는 프로그램을 HashMap을 이용하여 작성하시오. 프로그램은 고객의 이름과 포인트를 함께 저장 관리하는데, 포인트는 추가 될 때마다 누적하여 저장한다.

```
** 포인트 관리 프로그램입니다. **  
이름과 포인트 입력 >>이재문 40  
(이재문,40)  
이름과 포인트 입력 >>황기태 50  
(이재문,40) (황기태,50)  
이름과 포인트 입력 >>황기태 60  
(이재문,40) (황기태,110)  
이름과 포인트 입력 >>김남윤 30  
(이재문,40) (김남윤,30) (황기태,110)  
이름과 포인트 입력 >>이재문 20  
(이재문,60) (김남윤,30) (황기태,110)  
이름과 포인트 입력 >>그만  
포인트 관리 종료
```



Package 명 capitalGame 로 작성 할 것

Class 명 : Game, RunMain

문제5> 나라와 수도 맞추기 게임의 실행 예시는 아래와 같다.

이 프로그램을 HashMap<String, String>을 이용하여 작성하시오. '키' 는 나라 이름이고, '값' 은 수도이다.

```
*** 수도 맞추기 게임을 시작합니다. ***
입력 : 1, 퀴즈 : 2, 종료 : 3 >>1
현재 8개 나라의 수도가 입력되어 있습니다.
나라와 수도 입력9 >>러시아 모스크바
나라와 수도 입력10 >>한국 서울
한국은 이미 있습니다.
나라와 수도 입력10 >>그만
입력 : 1, 퀴즈 : 2, 종료 : 3 >>1
현재 9개 나라의 수도가 입력되어 있습니다.
나라와 수도 입력10 >>프랑스 파리
나라와 수도 입력11 >>그만
입력 : 1, 퀴즈 : 2, 종료 : 3 >>2
미국의 수도는 ?워싱턴 DC
아닙니다.!!
호주의 수도는 ?시드니
정답!!
한국의 수도는 ?그만
입력 : 1, 퀴즈 : 2, 종료 : 3 >>3
게임을 종료합니다.
```